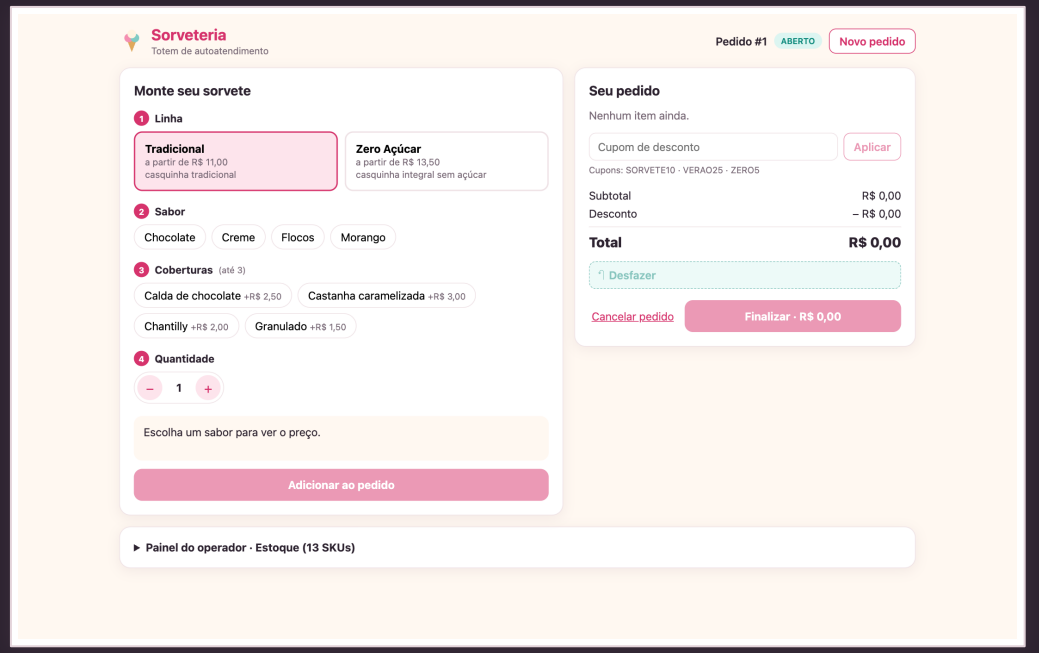
ARCHITECTURE CHALLENGE · PROJETOS DE ARQUITETURA DE SISTEMAS · UNIFOR

Totem da Sorveteria

Do console Java a 4 microsserviços
Spring Boot + React

Grupo 3
[Integrante 1] · [Integrante 2] · [Integrante 3]
[Integrante 4] · [Integrante 5]
Entrega final · 03/11/2026



Captura real do front-end rodando contra os 4 serviços

Mesmo domínio, arquitetura nova

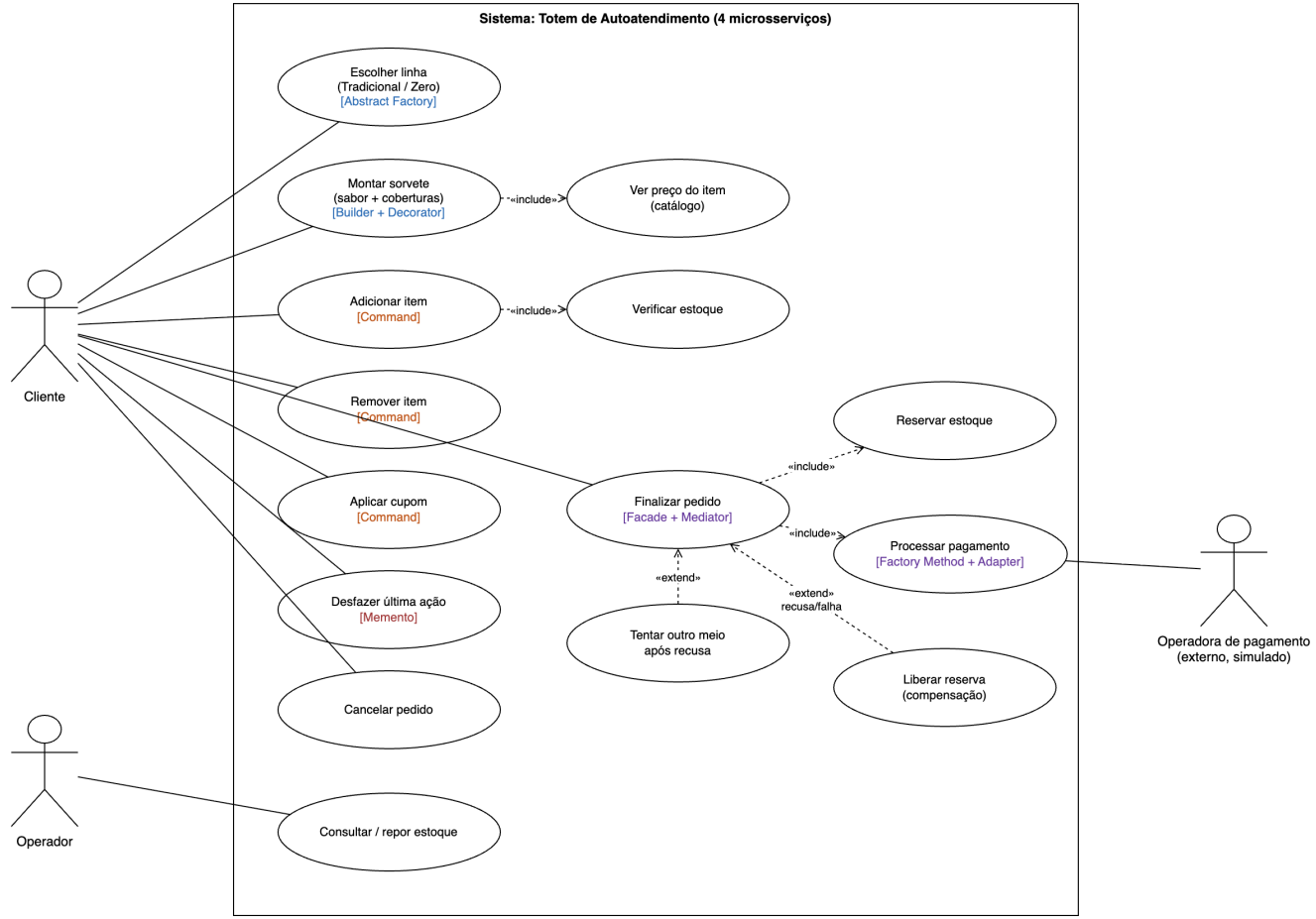
1. **Escolher a linha** — Tradicional / Zero
2. **Montar o sorvete** — sabor + até 3 coberturas
3. **Carrinho** — cupom e desfazer
4. **Pagar e retirar** — cartão ou Pix

	Projeto anterior	Architecture Challenge
Execução	Programa de console (Main.java)	4 microsserviços REST + front-end React
Padrões GoF	5: AF, Decorator, Command, Mediator, Memento	9 padrões — 3 de cada categoria GoF
Dinheiro	double	BigDecimal, arredondamento HALF_UP
Dados	Só em memória	Um banco H2 (JPA) por serviço
Estoque e pagamento	Não existiam	Reservas tudo-ou-nada + pagamento simulado
Testes	Nenhum	85 testes JUnit + 22 verificações ponta a ponta

Regra preservada: Zero morango + calda + granulado = R$ 17,50 — no projeto original e no novo (verificado por teste).

Funcionalidades e diagramas UML

Diagrama de Casos de Uso — Totem da Sorveteria (Architecture Challenge)



Arquivos editáveis (Draw.io + Mermaid) em docs/diagramas/

1 — Casos de uso
Cliente, Operador e Operadora de pagamento; cada caso indica o padrão que o implementa

2 — Classes
classes-padroes.drawio — 48 classes dos padrões nos 3 serviços

3 — Sequência
sequencia-checkout.drawio — checkout aprovado e bloco alt com recusa

4 — Estados
ABERTO → AGUARDANDO_PAGAMENTO → PAGO / RECUSADO / EM VERIFICAÇÃO

Arquitetura geral

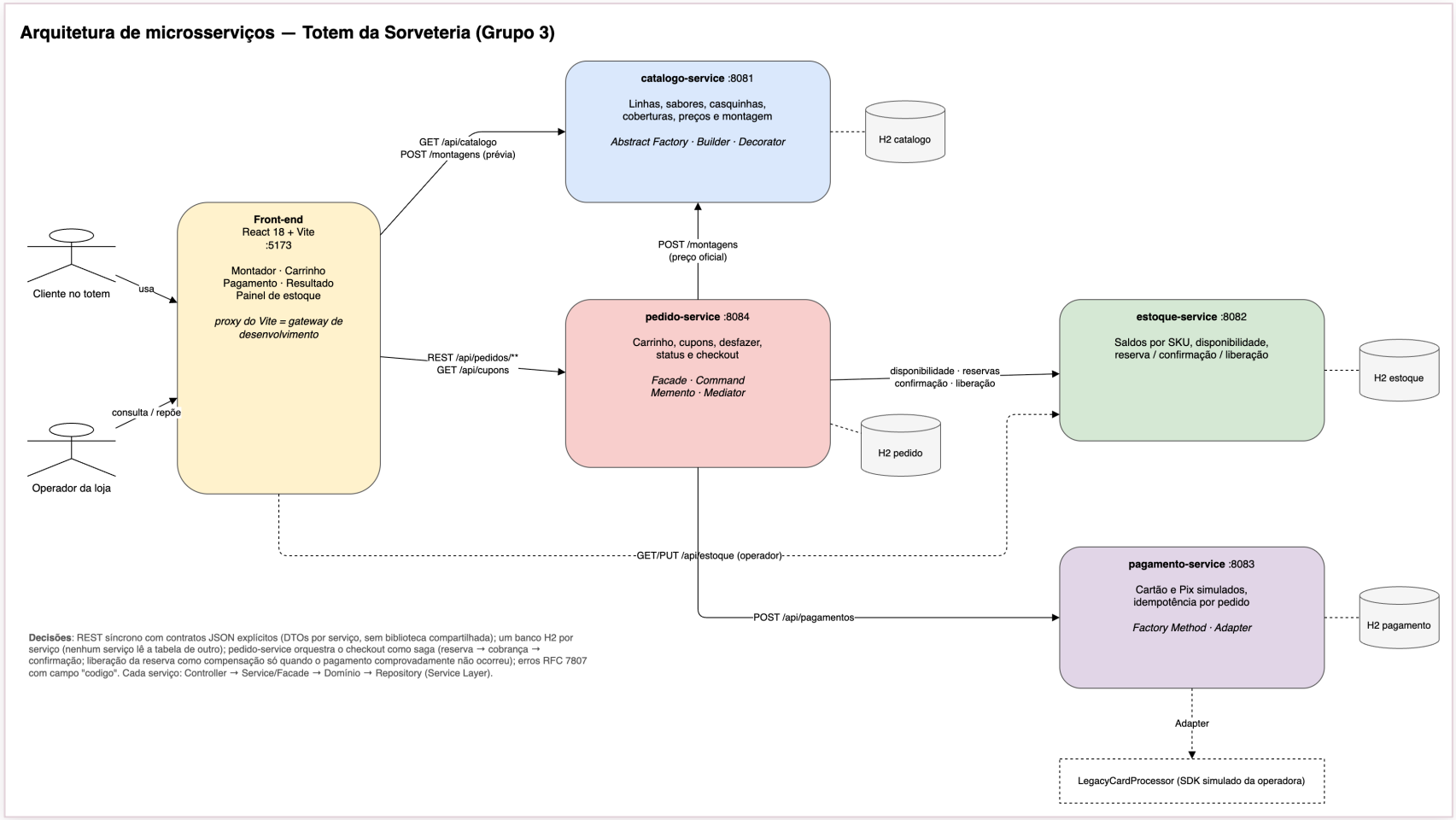


Diagrama para o MDL (mdlmodel.com): elementos e relações em docs/10-modelo-mdl.md

Um banco por serviço
Nenhum serviço lê a tabela de outro; o pedido pergunta o saldo à API do estoque.

REST com contratos
JSON explícito em ContratosRemotos.java; timeouts de 2 s e 5 s.

Service Layer
Controller → Service/Facade → Domínio → Repository em todos.

Os quatro microsserviços

:8081

catálogo-service

Linhas, sabores, coberturas, preços e montagem do sorvete

Dono de
itens_catalogo

No código
Abstract Factory · Builder · Decorator

:8084

pedido-service

Carrinho, cupons, desfazer, status e checkout

Dono de
pedidos, itens, cupons

No código
Facade · Command · Memento · Mediator

:8082

estoque-service

Saldos por SKU e reservas tudo-ou-nada

Dono de
itens_estoque, reservas

No código
Bloqueio otimista (@Version)

:8083

pagamento-service

Cartão e Pix simulados, sem cobrar duas vezes

Dono de
pagamentos (só 4 dígitos)

No código
Factory Method · Adapter

Cada serviço tem pom.xml, porta e banco próprios e roda sozinho: cd catalogo-service && mvn spring-boot:run

Comunicação REST e o checkout distribuído

1 **Reservar estoque** estoque-service

2 **Cobrar** pagamento-service

3 **Concluir** pedido-service

Aprovado confirma a reserva · pedido PAGO

Recusado libera a reserva (compensação) · pode tentar de novo

Incerto (timeout) mantém a reserva · PAGAMENTO_EM_VERIFICACAO, sem nova cobrança

O TotemCheckoutMediator orquestra a saga; a compensação só acontece quando o dinheiro comprovadamente não saiu.

Erro padronizado (RFC 7807) — resposta real ao cartão recusado

```
HTTP 402 Payment Required
{
  "codigo": "PAGAMENTO_RECUSADO",
  "detail": "Saldo insuficiente",
  "trilha": [
    "CHECKOUT_INICIADO",
    "ESTOQUE_RESERVADO",
    "PAGAMENTO_RECUSADO",
    "ESTOQUE_LIBERADO",
    "PEDIDO_ATUALIZADO" ],
  "pedido": { "status":
    "PAGAMENTO_RECUSADO" }
}
```

Mesmo formato nos 4 serviços: o front-end decide pelo campo "codigo".

Decisões arquiteturais (ADRs)

ADR-001

4 serviços por dono dos dados
Evita um monólito cortado em portas; cupons ficam no pedido.

ADR-002

Service Layer, não MVC
Não há views no back-end: a view é o React.

ADR-004

H2 por serviço
Zero instalação para a demo; trocar para PostgreSQL é configuração.

ADR-005

Desfazer em memória
O pedido persiste; o histórico de desfazer é da sessão do totem.

ADR-006

Saga com Mediator, sem mensageria
O cliente espera a resposta na tela; REST síncrono basta.

ADR-008

BigDecimal + idempotência
Totais exatos; o mesmo pedido nunca é cobrado duas vezes.

9 ADRs completos (contexto → decisão → consequências) em docs/05-adrs.md

Padrões criacionais

AF

Abstract Factory

catálogo · FabricaZero

Cria a família sorvete + casquinha da mesma linha: um sorvete Zero nunca sai numa casquinha tradicional.

```
// FabricaZero.java
Sorvete criarSorvete(
    ItemCatalogo sabor) {
  return new SorveteZero(
    sabor,
    criarCasquinha(),
    itens.buscar(SKU_BASE)
         .getPreco());
}
```

B

Builder

catálogo · SorveteBuilder

Monta o sorvete passo a passo e só entrega se for válido: sabor da linha, até 3 coberturas, sem repetir.

```
// MontagemService.java
new SorveteBuilder(cob)
  .daLinha(fabrica)
  .comSabor(morango)
  .comCobertura(calda)
  .comCobertura(granulado)
  .construir(); // valida
```

FM

Factory Method

pagamento · ProcessadorPagamento

O algoritmo de cobrança é fixo; cada subclasse decide qual gateway criar (cartão ou Pix).

```
// ProcessadorPagamento.java
final ResultadoGateway
    processar(s) {
  validar(s);
  GatewayPagamento g =
      criarGateway(); // FM
  return g.cobrar(s);
}
```

Padrões estruturais

D

Decorator

catálogo · CoberturaDecorator

Cada cobertura embrulha o sorvete e soma descrição, preço e item de estoque — combinações livres, sem subclasses.

```
// CoberturaDecorator.java
BigDecimal getPreco() {
  return sorvete.getPreco()
    .add(cobertura
         .getPreco());
}
// new Granulado(
//   new CaldaChocolate(s))
```

A

Adapter

pagamento · CartaoGatewayAdapter

Traduz o SDK legado da operadora (centavos, códigos "51", timeout) para a nossa interface GatewayPagamento.

```
// CartaoGatewayAdapter.java
resposta = adaptee.authorize(
  s.numeroCartao(),
  emCentavos(s.valor()),
  "BRL");
// 51 -> Saldo insuficiente
```

F

Facade

pedido · PedidoFacade

O controller conhece só a fachada; por trás há clientes HTTP, comandos, histórico, repositórios e o mediador.

```
// PedidoController.java
PedidoResponse adicionarItem(
    Long id,
    NovoItemRequest req) {
  return facade
    .adicionarItem(id, req);
}
```

Padrões comportamentais

C

Command

pedido · AdicionarItemCommand

Cada ação do carrinho vira um objeto que sabe executar e desfazer; o histórico é uma pilha por pedido.

```
// ComandoPedido.java
final void executar(
    Pedido receptor) {
  SnapshotPedido antes =
    receptor.criarSnapshot();
  aplicar(receptor);
  this.backup = antes;
}
```

Me

Memento

pedido · SnapshotPedido

Foto do pedido (itens + cupom) que só o próprio Pedido consegue ler; o comando guarda sem enxergar.

```
// ComandoPedido.java
void desfazer(
    Pedido receptor) {
  receptor.restaurar(
      backup);
}
// getters do snapshot:
// package-private
```

Md

Mediator

pedido · TotemCheckoutMediator

Estoque, pagamento e pedido não se conhecem: só avisam o mediador, que decide o próximo passo.

```
// TotemCheckoutMediator
case PAGAMENTO_RECUSADO -> {
  ctx.concluir(RECUSADO,
    ctx.getMensagem());
  estoque.liberar(ctx);
  pedido.registrarRecusa(
      ctx);
}
```

SOLID e GRASP no código

SOLID

S — **Responsabilidade única**
SorveteBuilder só monta; PedidoController só traduz HTTP

O — **Aberto/Fechado**
Nova linha = nova FabricaLinha; FabricasPorLinha não muda

L — **Liskov**
Qualquer CoberturaDecorator substitui um Sorvete

I — **Segregação de interfaces**
Um cliente por serviço: CatalogoClient, EstoqueClient, PagamentoClient

D — **Inversão de dependência**
Fábricas dependem da porta CatalogoItens, não do JPA

GRASP

Information Expert
Pedido.getTotal(), Reserva.liberar(), Cupom.aplicavelEm()

Creator
FabricaZero cria SorveteZero; Pedido cria SnapshotPedido

Controller
Controllers REST delegam; o Mediator controla o checkout

Baixo acoplamento
Serviços se conhecem só por JSON; participantes não se conhecem

Alta coesão
Um assunto por serviço e um pacote por padrão

Polimorfismo
TipoDesconto.calcular(); criarGateway() — sem if/switch por tipo

Demonstração ao vivo

1. **Zero Açúcar · Morango · Calda · Granulado**
   prévia R$ 17,50 vinda do catálogo

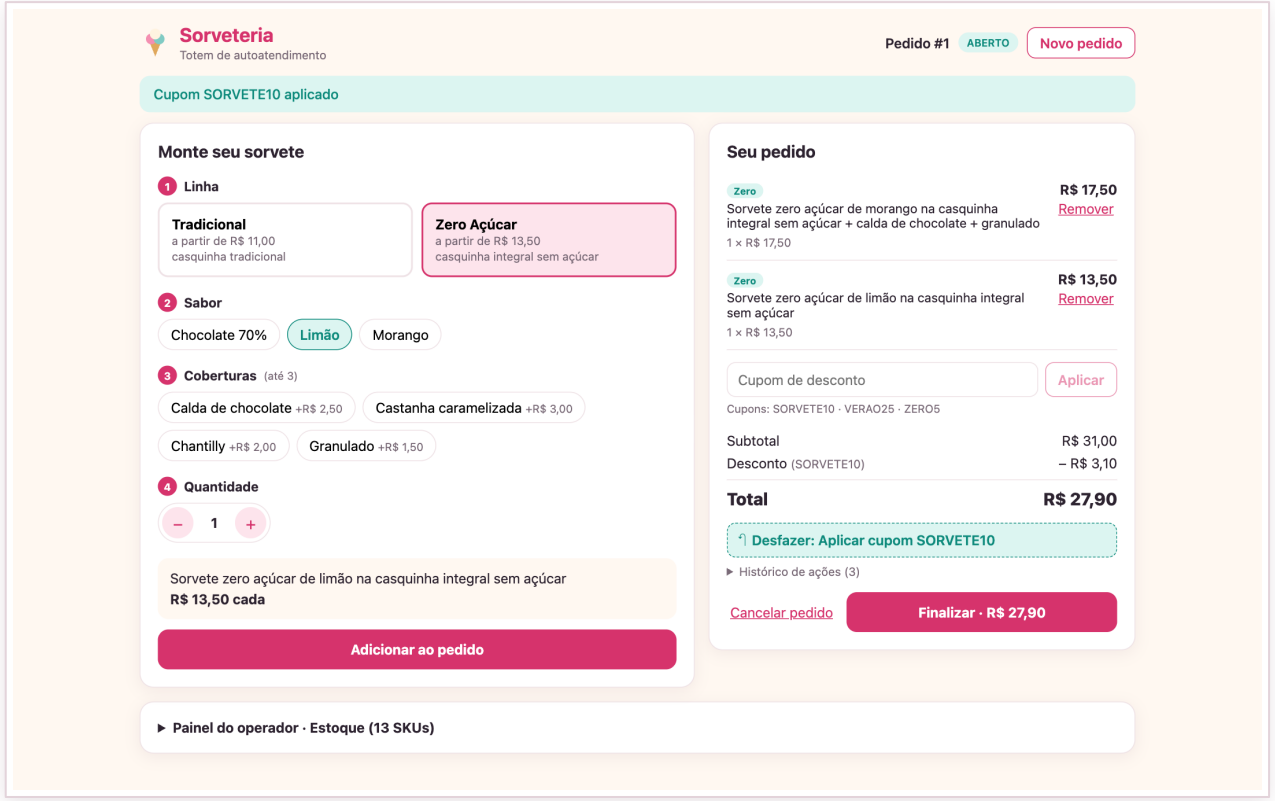
2. **Marcar 4 coberturas**
   erro do Builder: máximo de 3

3. **Adicionar · Limão · Adicionar**
   total R$ 31,00

4. **Cupom SORVETE10 → Desfazer**
   R$ 27,90 → R$ 31,00

5. **Finalizar · Cartão recusado**
   402, trilha com ESTOQUE_LIBERADO

6. **Finalizar · Cartão aprovado**
   pedido PAGO, autorização



Captura real do passo 4 (cupom aplicado, botão Desfazer mostra o próximo comando)

Plano B: capturas em ENTREGA_FINAL/capturas e ./scripts/smoke-test.sh

Testes automatizados e resultados

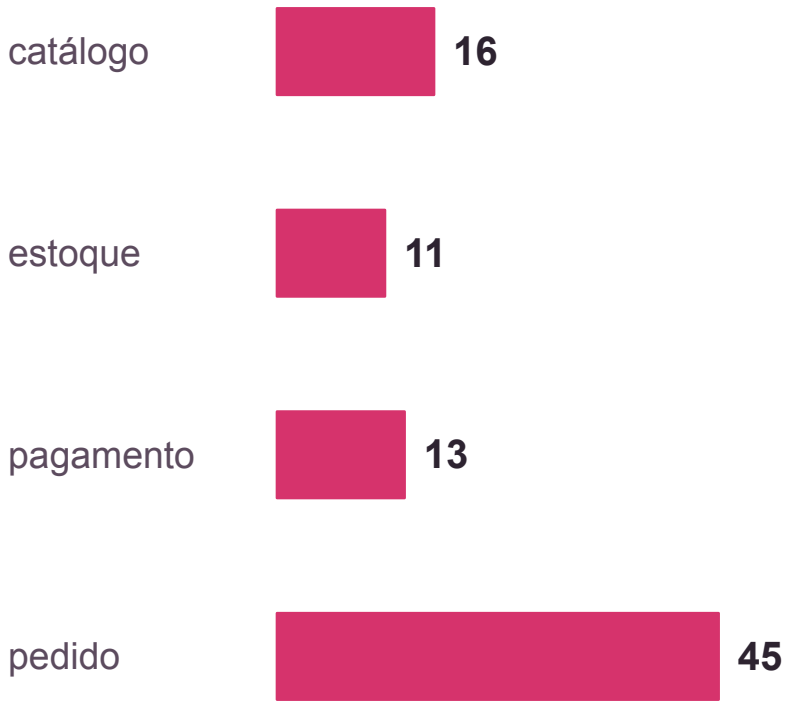
85

testes JUnit 5 · 0 falhas

22/22

verificações ponta a ponta com os 4 serviços reais

Testes por serviço (mvn test)



Unitários
padrões e regras (BigDecimal, cupons, Builder)

Integração
MockMvc + H2 sobem cada serviço inteiro

Contrato HTTP
MockRestServiceServer: JSON enviado e erros

Concorrência
2 cobranças simultâneas → 1 pagamento

Execuções reais registradas em ENTREGA_FINAL/evidencias (mvn-test_resumo.txt, smoke-test_saida.txt)

Limitações conhecidas e próximos passos

Limitações (documentadas)

- **Dados em memória**
  H2 reinicia a cada execução (ADR-004)
- **Desfazer da sessão**
  histórico some se o serviço reiniciar (ADR-005)
- **Pagamento em verificação**
  sem aprovação registrada, exige decisão manual
- **Uma instância**
  a trava contra cobrança dupla vale para uma réplica
- **Sem autenticação**
  totem anônimo; sem API Gateway em produção

Próximos passos (backlog)

B21 Expirar reservas pendentes

B22 Persistir o histórico de desfazer

B23 Testes de front-end e E2E no navegador

B24 PostgreSQL + migrações (Flyway)

B28 Trava de pagamento no banco (várias réplicas)

B29 Tela do atendente para pedidos em verificação

Conclusão

1

Evolução real
O mesmo totem, agora em 4 microsserviços com banco próprio e um front-end React.

2

Padrões no fluxo
Os 9 padrões aparecem no caminho real de uma compra — e têm testes.

3

Falhas tratadas
Recusa, falta de estoque, serviço fora do ar e resultado incerto têm resposta definida.

Obrigado! Perguntas?

Código, testes, diagramas e documentação: README.md e pasta docs/ do projeto